

Binary Search

Matvei Gavrilenko

February 2026

Code 1:

```
1 #include <iostream>
2 #include <vector>
3
4 using namespace std;
5
6 bool binary_search(const vector<int32_t>& numbers, const int32_t& request)
7 {
8     int32_t left = 0;
9     auto right = static_cast<int32_t>(numbers.size());
10
11     int32_t mid = 0;
12     while (left < right) {
13         mid = (right + left) / 2;
14
15         if (numbers[mid] == request) {
16             return true;
17         }
18         else if (numbers[mid] < request) {
19             left = mid;
20         }
21         else {
22             right = mid;
23         }
24     }
25
26     return false;
27 }
28
29 int main() {
30     int32_t n = 0;
31     cin >> n;
32
33     vector<int32_t> numbers(n);
34     int32_t number = 0;
35     for (size_t i = 0; i < n; i++) {
36         cin >> number;
37         numbers[i] = number;
38     }
39
40     int32_t k = 0;
41     cin >> k;
42
43     const bool b = binary_search(numbers, k);
44
45     cout << b << endl;
46
47     return 0;
48 }
```

Console Output

```
1 user1@matvei-HP-Laptop-15s-eq2xxx:~/Programming/Algorithms/Binary_search/  
  task_0.1_binary_search$ ./main  
2 1  
3 1  
4 2  
5 ...
```

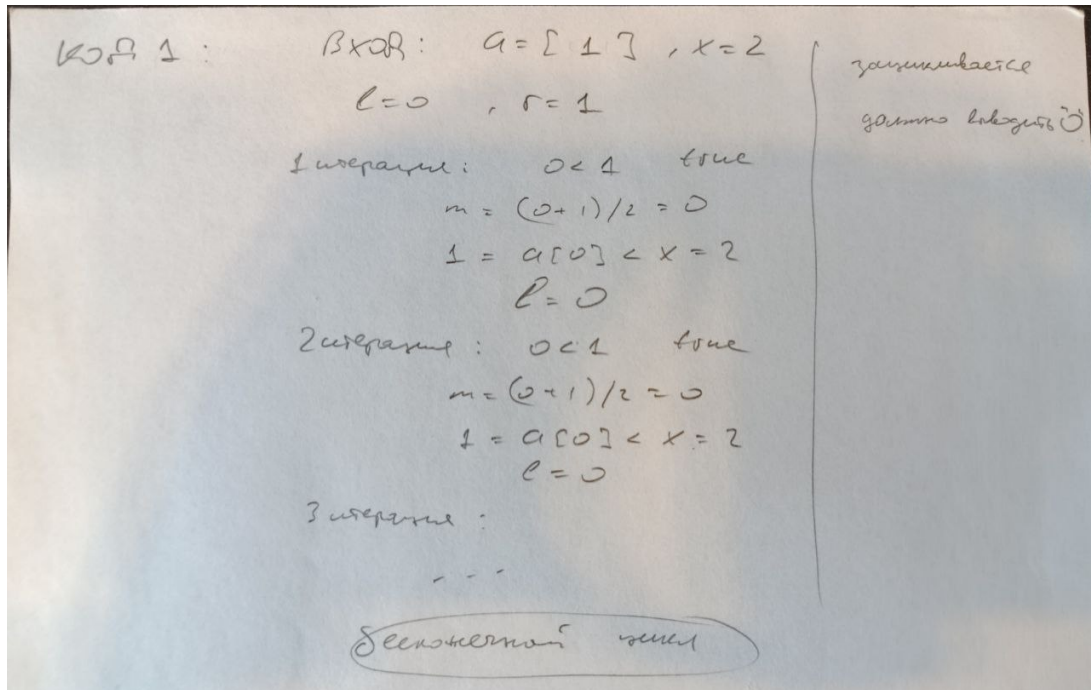


Рис. 1: Visualization of binary search algorithm

Code 2:

```
1 #include <iostream>
2 #include <vector>
3
4 using namespace std;
5
6 bool binary_search(const vector<int32_t>& numbers, const int32_t& request)
7 {
8     int32_t left = 0;
9     auto right = static_cast<int32_t>(numbers.size());
10
11     int32_t mid = 0;
12     while (left < right) {
13         mid = (right + left) / 2;
14
15         if (numbers[mid] == request) {
16             return true;
17         }
18         else if (numbers[mid] < request) {
19             left = mid + 1;
20         }
21         else {
22             right = mid - 1;
23         }
24     }
25
26     return false;
27 }
28
29 int main() {
30     int32_t n = 0;
31     cin >> n;
32
33     vector<int32_t> numbers(n);
34     int32_t number = 0;
35     for (size_t i = 0; i < n; i++) {
36         cin >> number;
37         numbers[i] = number;
38     }
39
40     int32_t k = 0;
41     cin >> k;
42
43     const bool b = binary_search(numbers, k);
44
45     cout << b << endl;
46
47     return 0;
48 }
```

Console Output

```
1 user1@matvei-HP-Laptop-15s-eq2xxx:~/Programming/Algorithms/Binary_search/  
  task_0.1_binary_search$ ./main  
2 2  
3 0 1  
4 0  
5 0
```

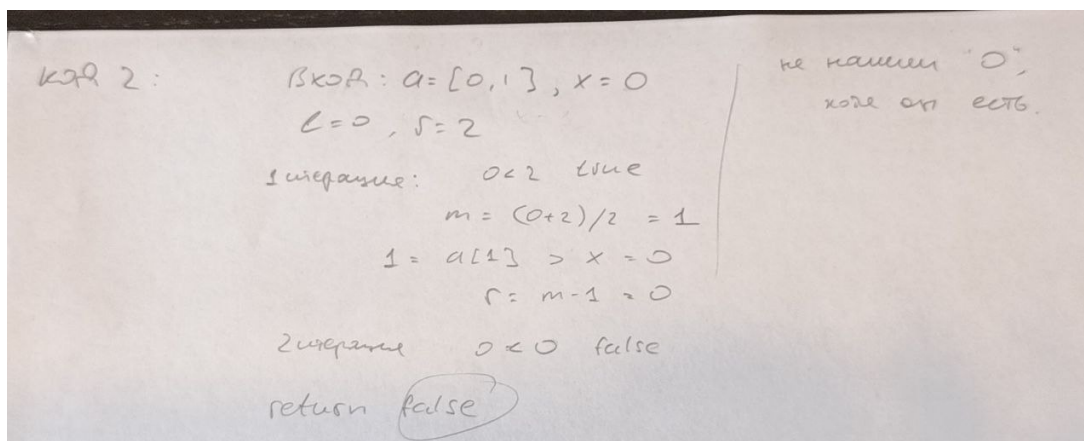


Рис. 2: Visualization of binary search algorithm

Code 3:

```
1 #include <iostream>
2 #include <vector>
3
4 using namespace std;
5
6 bool binary_search(const vector<int32_t>& numbers, const int32_t& request)
7 {
8     int32_t left = 0;
9     auto right = static_cast<int32_t>(numbers.size());
10
11     int32_t mid = 0;
12     while (left < right) {
13         mid = (right + left + 1) / 2;
14
15         if (numbers[mid] == request) {
16             return true;
17         }
18         else if (numbers[mid] < request) {
19             left = mid + 1;
20         }
21         else {
22             right = mid;
23         }
24     }
25
26     return false;
27 }
28
29 int main() {
30     int32_t n = 0;
31     cin >> n;
32
33     vector<int32_t> numbers(n);
34     int32_t number = 0;
35     for (size_t i = 0; i < n; i++) {
36         cin >> number;
37         numbers[i] = number;
38     }
39
40     int32_t k = 0;
41     cin >> k;
42
43     const bool b = binary_search(numbers, k);
44
45     cout << b << endl;
46
47     return 0;
48 }
```

Console Output

```
1 user1@matvei-HP-Laptop-15s-eq2xxx:~/Programming/Algorithms/Binary_search/  
  task_0.1_binary_search$ ./main  
2 2  
3 0 1  
4 0  
5 ...
```

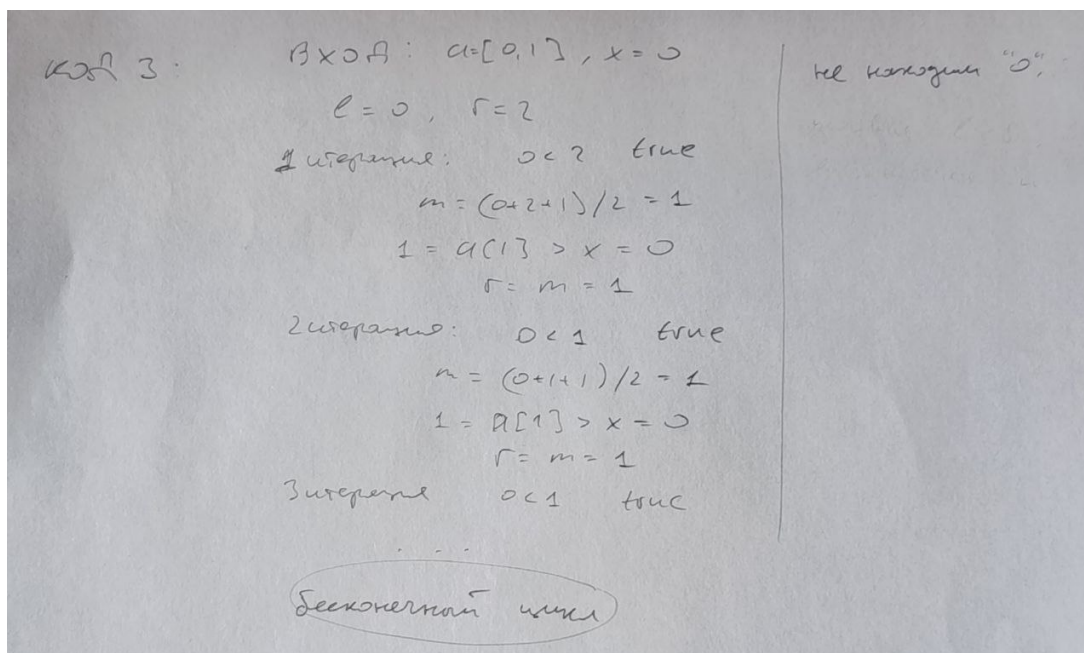


Рис. 3: Visualization of binary search algorithm

My code:

```
1 #include <iostream>
2 #include <vector>
3
4 using namespace std;
5
6 bool binary_search(const vector<int32_t>& numbers, const int32_t& request)
7 {
8     int32_t left = 0;
9     auto right = static_cast<int32_t>(numbers.size());
10
11     int32_t mid = 0;
12     while (left < right) {
13         mid = left + (right - left) / 2;
14
15         if (numbers[mid] == request) {
16             return true;
17         }
18         else if (numbers[mid] < request) {
19             left = mid + 1;
20         }
21         else {
22             right = mid;
23         }
24     }
25
26     return false;
27 }
28
29 int main() {
30     int32_t n = 0;
31     cin >> n;
32
33     vector<int32_t> numbers(n);
34     int32_t number = 0;
35     for (size_t i = 0; i < n; i++) {
36         cin >> number;
37         numbers[i] = number;
38     }
39
40     int32_t k = 0;
41     cin >> k;
42
43     const bool b = binary_search(numbers, k);
44
45     cout << b << endl;
46
47     return 0;
48 }
```

Все нюансы и неточности мы учли, код работает верно!